

Q-Learning

Naf Guo
2026

Q-function

Two types of Value-Based Methods

Action Value Function:

$$Q_{\pi}(s, a) = \mathbf{E}_{\pi}[G_t | S_t = s, A_t = a]$$

Value of state-action pair s, a

Expected return

If the agent starts at state s

and chooses action a

And then uses the policy to choose its actions for all time steps

For each state and action, the action-value function outputs the expected return if the agent starts in that state and takes the action and then follows the policy forever after.

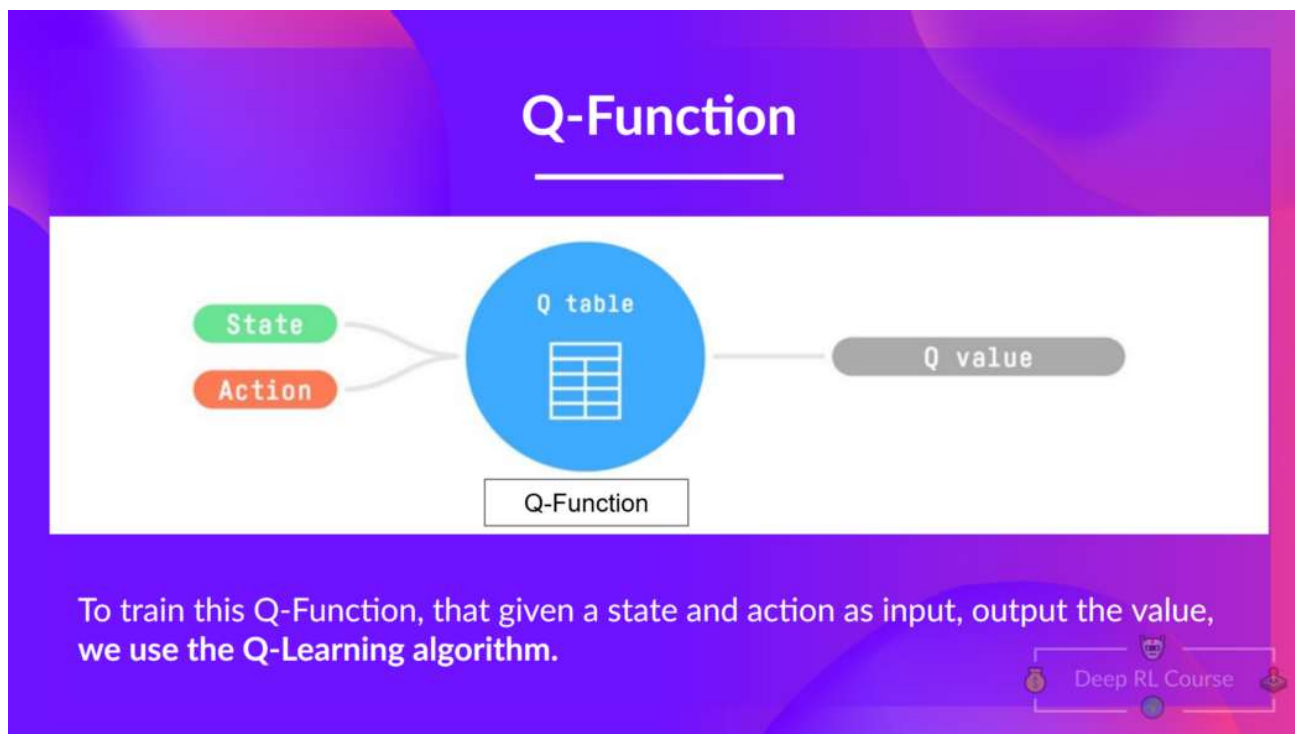
Deep RL Course

之前的蒙特卡洛方法和时序差分法都用的是状态价值函数，或者V函数；

我们也可以用动作价值函数，或称Q函数来进行学习。

Q函数是指，目前处在 s 态，且下一步采用动作 a ，之后再按策略 π 完成整个游戏/过程得到的期望回报，用这个回报作为状态 s ，动作 a 的价值

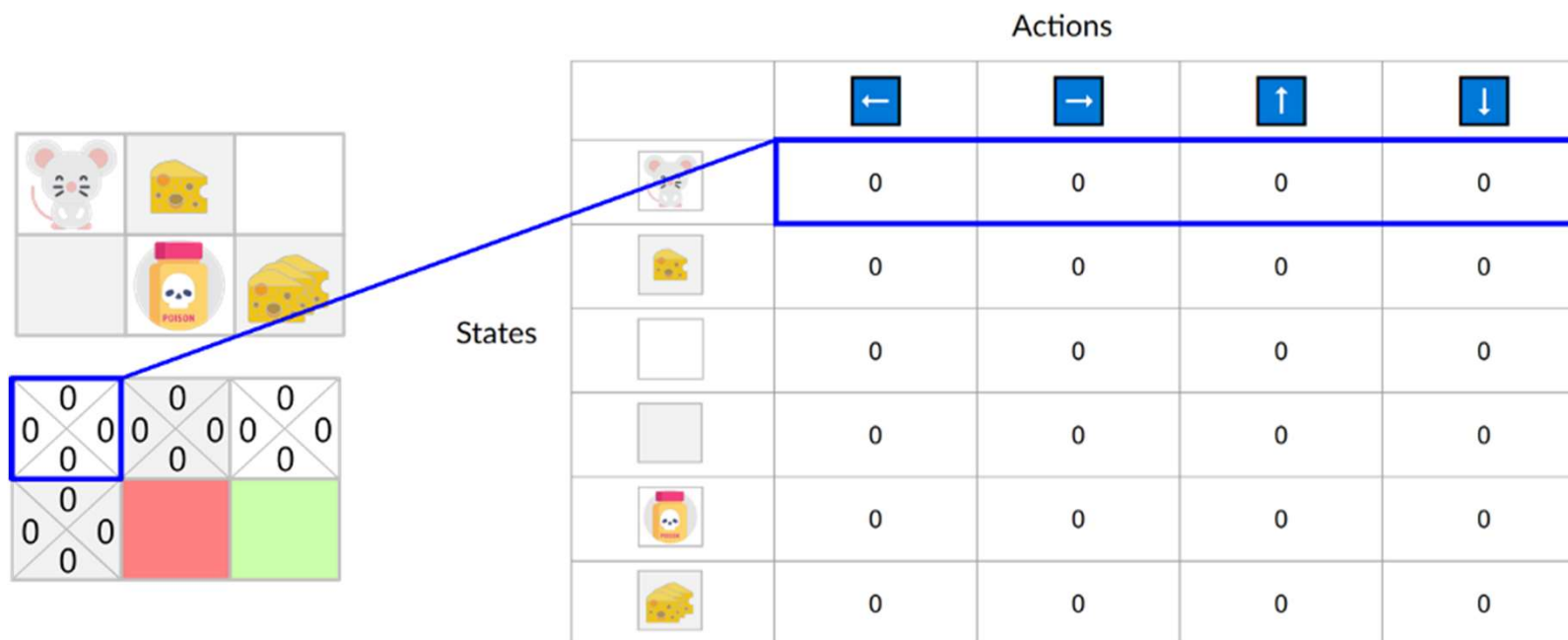
Q-learning



如何基于Q函数进行学习呢？

简单的说我们要维护一张Q表格，对所有的状态-动作对，都能查表得到对应的Q函数值，也就是某状态下某动作的价值。

Q-learning



如图所示，小老鼠这次需要吃到奶酪且避开毒药。我们看到右边的Q表格，状态对应6个格点，动作对应可以采取的4个运动方向，那Q-学习就是要获得每个格点下4个动作的价值。

Q-learning

If we recap, *Q-Learning* is the RL algorithm that:

- Trains a *Q-function* (an **action-value function**), which internally is a **Q-table** that contains all the state-action pair values.
- Given a state and action, our Q-function **will search its Q-table** for the corresponding value.
- When the training is done, **we have an optimal Q-function**, which means we have optimal Q-table.
- And if we **have an optimal Q-function**, we **have an optimal policy** since we **know the best action to take** at each state.

The link between Value and Policy:

$$\pi^*(s) = \arg \max_a Q^*(s, a)$$

Finding an optimal value function leads to having an optimal policy.

在Q学习中，我们要做的是训练一套动作-状态函数，对每一组动作-状态，都给出一个Q值（也就是在该状态下采取该动作能获得的期望回报）

之后每次行动，都根据当前状态采取Q值最大的行动

一套最优的Q函数能使我们获得最大的总回报

依据该套Q函数行动的策略就是最优策略

Q-learning

Q-Learning, Step 1

Initialize Q arbitrarily (e.g., $Q(s, a) = 0$ for all $s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$, and $Q(\text{terminal-state}, \cdot) = 0$)

	←	→	↑	↓
	0	0	0	0
	0	0	0	0
	0	0	0	0
	0	0	0	0
	0	0	0	0
	0	0	0	0

We initialize the Q-Table

首先初始化Q表格，一开始都设为0。

Q-learning

Q-Learning, Step 2

Choose action A_t using policy derived from Q (e.g., ϵ -greedy)



Choose the action using ϵ -greedy policy

之后要采取行动，取一个参数 ϵ ，以 ϵ 的概率随机采取行动，否则采取贪婪行动，或者说对应最大Q值的行动。

这是为了在探索和利用之间达到一个平衡。

Q-learning

Q-Learning, Step 3

Take action A_t and observe R_{t+1}, S_{t+1}

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)]$$

New
Q-value
estimation

Former
Q-value
estimation

Learning
Rate

Immediate
Reward

Discounted Estimate
optimal Q-value
of next state

Former
Q-value
estimation

TD Target

TD Error

采取行动以后，会获得当步奖励 R_{t+1} ，到达下一个状态 S_{t+1} ，之后根据左边的式子更新Q表格。

可以看到，更新的公式也用的时序差分法，只不过价值函数换成了Q函数。

要注意，TD目标里，对下一个状态 S_{t+1} ，使用的所有动作里能使 $Q(S_{t+1}, a)$ 最大的那个。

这里的含义是，从状态 S_t 出发，采取动作 A_t ，获得的奖励 R_{t+1} ，并且到达状态 S_{t+1}

而在贪婪策略下

从状态 S_t 出发，采取动作 A_t 获得的期望回报 = 采取行动 A_t 获得的奖励 R_{t+1} + 从状态 S_{t+1} 采取最优动作获得的期望回报